Evaluation of Role Segregation in Language Models via Introspection of Thought

Daniil Privezentsev[1*]

[1*]Higher School of Economics, Moscow, Russia.

Corresponding author(s). E-mail(s): daprivezentsev@edu.hse.ru;

Abstract

As agentic workflows built on top of large language models become more common, inference cost, context growth, and the reliability of intermediate decisions become major constraints for practical deployment in software engineering. This paper presents a scientifically grounded reworking of the original problem statement on role segregation using Introspection of Thought (INoT). Unlike draft versions of the manuscript, the paper does not treat unverified architectural claims as facts and does not fabricate experimental results. Its contribution is fourfold: (i) a literature audit that removes weak and unverifiable sources; (ii) a formalization of a hybrid agent architecture in which *planner*, *worker*, *critic*, *self-check*, *memory/compression*, and *policy controller* roles are coordinated around an internal introspective loop; (iii) a system of metrics for quality, token efficiency, monetary cost, reliability, and code maintainability, including *utility-per-token*, *quality-per-dollar*, *verified-code-rate*, and *maintainability-aware success*; and (iv) a reproducible experimental protocol together with a synthetic-but-reproducible total-cost-of-ownership section. Analytically, the proposed architecture is advantageous over externally orchestrated multi-agent workflows when savings from reduced context replay and fewer expensive reruns exceed the overhead of internal critique, compression, and self-check.

Keywords: Large Language Models, AI Agents, Introspection of Thought, Software Engineering, Token Efficiency, Economics of Inference, Code Reliability

1 Introduction

Modern software development requires constant switching among source code, version control, issue trackers, documentation, and automated checks. Empirical studies show

that such context switching increases cognitive load and delays task resumption [2–4]. LLM-based assistants are often framed as a way to mitigate these costs, yet the move from chat-style assistance to full agentic workflows introduces a new bottleneck: functional capability improves, but token expenditure, latency, and orchestration overhead rise as well.

Classical LLM agents are commonly implemented as externally orchestrated think–act–observe loops with tool calls, delegated roles, and repeated invocations of a large model [6, 17]. This design is valuable when tool isolation and real interaction with the environment are necessary, but it also creates repeated transmission of long context across roles, duplicated intermediate artifacts, and cascading inference costs. For tasks such as code review, local refactoring, document reasoning, and change justification, this architecture may be heavier than necessary.

One possible response is to move part of role interaction inside a single model context. Test-time reasoning improvement through intermediate steps is well established in Chain-of-Thought, ReAct, Self-Refine, and Reflexion [5–8]. Sun and Zeng extend this direction with *Introspection of Thought* (INoT), where one model simulates several roles and performs an internal cycle of generation, critique, and correction [1]. Yet introspection alone does not settle the methodological issues. Intermediate text should not be treated as a faithful explanation of the model's latent computation [9], and the literature still lacks a rigorous formulation that ties internal role segregation specifically to inference economics, code reliability, and maintainability in agentic software-engineering pipelines.

The central research question of this paper is therefore the following: can an agent architecture with internal role introspection increase utility per token and utility per dollar without degrading quality or engineering reliability relative to both a single-shot single-agent setup and a classically orchestrated multi-agent workflow?

The paper contributes a source audit, a formal hybrid architecture with *planner*, *worker*, *critic*, *self-check*, *memory/compression*, and *policy controller* roles, an analytical cost model, new efficiency and reliability metrics, and a reproducible evaluation and TCO protocol.

2 Related Work and Positioning

2.1 From step-by-step reasoning to introspective test-time improvement

Wei et al. showed that explicitly eliciting intermediate reasoning steps can improve performance on complex tasks [5]. ReAct combines reasoning and acting in an external environment [6]. Self-Refine moves generation, feedback, and revision into a single-model loop [7], while Reflexion adds verbal feedback and reflective memory to agentic workflows [8]. Sun and Zeng place INoT on this line as a more structured form of internal multi-role debate [1]. In our setting, the key point is operational rather than introspective in the philosophical sense: intermediate artifacts are treated as controllable test-time objects for generation, filtering, and rerun decisions, not as a transparent view into hidden cognition.

2.2 Multi-agent systems for software engineering

The classical theoretical basis for multi-agent systems comes from Weiss and Wooldridge [10, 11]. In the LLM era, role-specialized systems such as MetaGPT and ChatDev show how architect, programmer, and tester roles can be separated to improve task decomposition [12, 13]. At the same time, surveys emphasize the costs of coordination, error propagation across roles, and the difficulty of scaling collaboration [14, 15]. In software engineering, this is especially relevant when multiple roles repeatedly consume the same long code context. Benchmarks such as SWE-bench and systems such as SWE-agent further show that real software-engineering automation requires long context, file edits, tests, and multiple iterations [16, 17].

2.3 Context compression, code self-check, and maintainability

For long contexts, more tokens mean not only higher cost but also more noise. LongLLMLingua and LLMLingua-2 show that prompt compression can reduce latency while maintaining or even improving performance in long-context settings [20, 21]. In software engineering, Jia et al. directly connect context compression to issue resolution and demonstrate large token savings on SWE-bench-style tasks [22]. Code self-check should also not be reduced to textual reflection. Compiler feedback, tests, and static analysis remain necessary external signals [23–25].

2.4 Publicly confirmed evidence about contemporary systems

In addition to academic literature, the paper must distinguish between publicly confirmed facts and speculation about hidden product internals. This matters especially for ‘DeepSeek‘, ‘GLM-4.5‘, and ‘OpenAI/ChatGPT‘.

2.5 Research gap and novelty

Existing work covers either external multi-agent coordination or sequential self-improvement within one model. The underexplored middle ground is a *hybrid agent architecture* in which most role interaction happens inside one context while specialized external procedures are used only selectively for compression, verification, and tool control. The novelty of the present paper lies not in introducing intermediate reasoning per se, but in formalizing it as a mechanism for managing inference economics and engineering quality in agentic software engineering.

3 Problem Statement, Hypotheses, and Formal Setup

Let $x \in \mathcal{X}$ denote a task, $\mathcal{C}_0$ its raw context, and $\mathcal{Z}$ the set of allowed tools. The system must produce an answer or patch y that maximizes quality $Q(y, x)$ while minimizing token usage T, monetary cost C, and the risk of unreliable code changes.

We distinguish three architectural families: **Single-Large**, **Classical-MAS**, and **Hybrid-INoT**. Hypothesis **H1** states that with long shared context, Hybrid-INoT achieves higher *utility-per-token* than Classical-MAS by reducing repeated context transmission. Hypothesis **H2** states that small helper models for compression and

Table 1 Publicly confirmed and unconfirmed claims about contemporary systems

System	Confirmed by primary sources	Unconfirmed and therefore not used as fact
DeepSeek-V3	The technical report confirms an MoE backbone, Multi-head Latent Attention, and a cost-efficiency-oriented design [26].	No public primary source confirms an internal multi-agent architecture or INoT-style role debate.
DeepSeek-R1	The technical report confirms reasoning-oriented post-training and RL-based improvement in the R1 line [27].	No public evidence confirms a specific hidden multi-role orchestration stack in product agents.
GLM-4.5	Public sources confirm the model's agentic/reasoning/coding orientation and the existence of a technical report [28].	It is not publicly confirmed that GLM-4.5 uses internal multi-agent debate as its core computation scheme.
OpenAI / ChatGPT agent	OpenAI publicly confirms a unified agentic product with browser, terminal, connectors, and a system card [29, 30]. OpenAI also publishes SDK-level handoff patterns and agent workflows [31, 32].	The internal role decomposition of ChatGPT agent itself is not publicly documented beyond product-level description, so it is not used as an established fact.

self-check are economically justified if they reduce the probability of an expensive rerun of the large model more than they cost themselves. Hypothesis **H3** states that the advantage of Hybrid-INoT weakens when tasks require genuine tool parallelism, privilege isolation, or prolonged external interaction.

4 Method

4.1 Role architecture and data flow

The proposed system uses the roles *policy controller*, *memory/compression*, *planner*, *worker*, *critic*, and *self-check*. First, the *policy controller* sanitizes the input and checks whether tool actions are permitted. Next, *memory/compression* extracts a relevant context $\hat{\mathcal{C}} \subseteq \mathcal{C}_0$. The *planner* then chooses either an internal introspective mode or an external agent mode. Inside the introspective mode, the *worker* produces K candidates, the *critic* scores them for utility and risk, and *self-check* either accepts the best candidate or triggers a selective rerun only for those components that fail verification.

4.2 Formal notation

We use the following notation:

$$x \in \mathcal{X} \qquad \text{task},$$
$$g(x) \in \mathcal{G} \qquad \text{task class},$$

$\mathcal{C}_0$	raw context,
$\hat{\mathcal{C}}$	compressed context,
K	number of candidates,
I	maximum number of iterations,
p_i^L, r_i^L	large-model input and output tokens,
p_i^S, r_i^S	small-model input and output tokens,
c_L^{in}, c_L^{out}	large-model token prices,
c_S^{in}, c_S^{out}	small-model token prices,
ρ	probability of an expensive rerun,
Q	normalized task quality in $[0, 1]$.

The total token cost of a trajectory is

$$T = \sum_{i=1}^{I}(p_i^L + r_i^L) + \sum_{i=1}^{I}(p_i^S + r_i^S) + T_{\text{tool}}. \tag{1}$$

For a classically orchestrated multi-agent workflow:

$$T_{\text{MAS}} = \sum_{i=1}^{I} \sum_{a \in \mathcal{A}} (p_{i,a} + r_{i,a}) + D_{\text{ctx}} + H_{\text{coord}}, \tag{2}$$

where D_{ctx} is the cost of repeatedly replaying shared context across agents and H_{coord} is external coordination overhead.

For the proposed Hybrid-INoT architecture:

$$T_{\text{INoT}} = T_{\text{core}} + T_{\text{cmp}} + T_{\text{check}} + T_{\text{rerun}}, \tag{3}$$

where T_{core} is the internal debate round, T_{cmp} is compression overhead, T_{check} is selective verification, and T_{rerun} is the cost of rerunning only after failed verification.

$$\mathbb{E}[T_{\text{MAS}} - T_{\text{INoT}}] > 0 \Longleftrightarrow \mathbb{E}[D_{\text{ctx}} + H_{\text{coord}}] > \mathbb{E}[T_{\text{cmp}} + T_{\text{check}} + T_{\text{rerun}} - T_{\text{core}}^{\text{saved}}], \tag{4}$$

4.3 Monetary cost and the small-model profitability condition

Inference cost is defined as

$$C_{\text{inf}} = c_L^{in} \sum_i p_i^L + c_L^{out} \sum_i r_i^L + c_S^{in} \sum_i p_i^S + c_S^{out} \sum_i r_i^S. \tag{5}$$

If a small model is used only as a self-check filter, it is economically justified when it reduces the expected cost of rerunning the large model:

$$C^{S}_{\text{check}} < (\rho_0 - \rho_1)\, C^{L}_{\text{rerun}}, \tag{6}$$

where ρ_0 and ρ_1 are rerun probabilities without and with the small-model self-check and C^{L}_{rerun} is the expected cost of such a rerun.

4.4 Workflow and algorithms

4.4.1 Workflow visualization

Figure 1 presents the main Hybrid-INoT loop as a UML activity diagram. Relative to pseudocode alone, the diagram makes three decision points explicit: routing between internal and external execution, passing versus failing self-check, and rerunning only while iteration budget remains.

4.4.2 Algorithmic specification

Algorithm 1 Hybrid introspective loop

Require: task x, context $\mathcal{C}_0$, token budget B, maximum iterations I
Ensure: final solution $y^\star$

```
C ← PolicySanitize(x, C_0)
Ĉ ← CompressIfNeeded(C)
m ← RouteMode(x, Ĉ)
if m = external then
    return ExternalAgentWorkflow(x, Ĉ)
end if
for i = 1 to I do
    π_i ← Planner(x, Ĉ, B)
    Y_i ← GenerateCandidates(π_i, K)
    S_i ← CriticScore(Y_i)
    y_i ← SelectBest(Y_i, S_i)
    v_i ← SelfCheck(y_i)
    if v_i = pass then
        return y_i
    else
        Ĉ ← UpdateContextWithFailures(Ĉ, v_i)
    end if
end for
return FallbackBestCandidate()
```

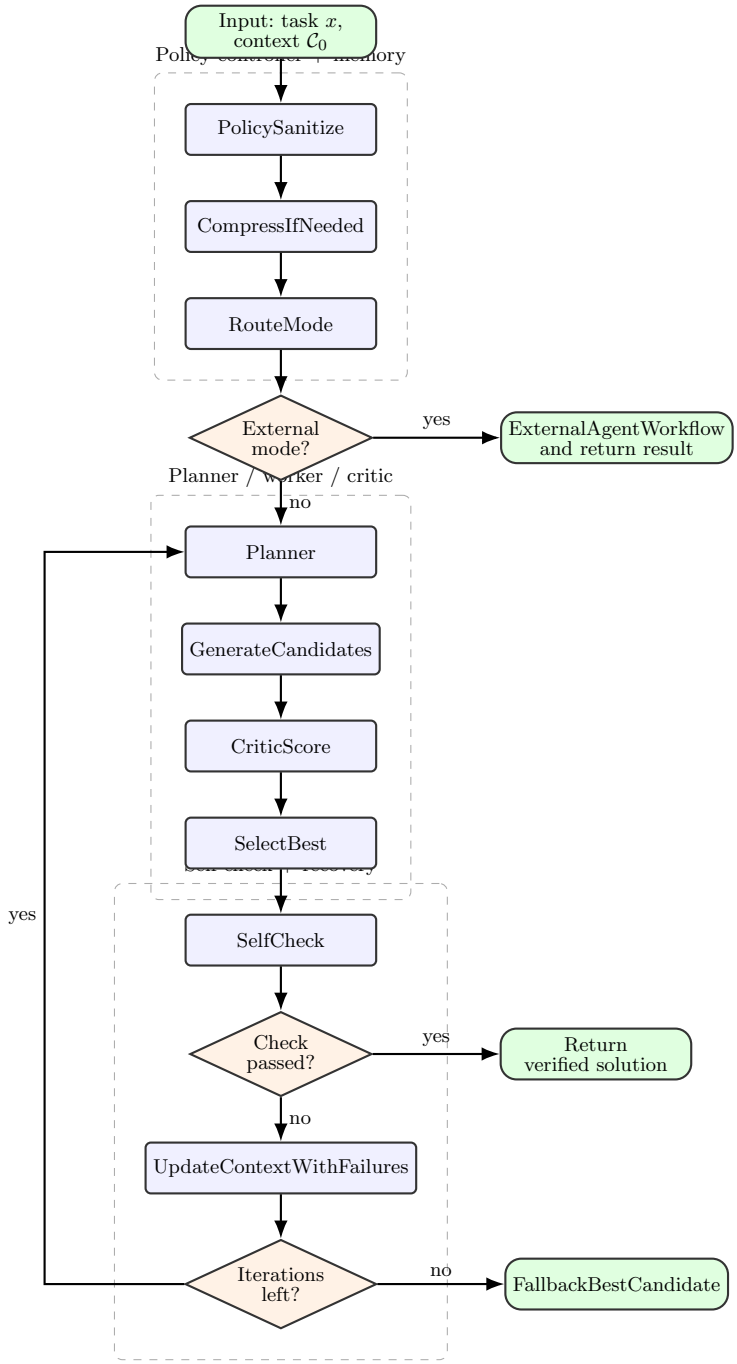

Fig. 1 UML activity diagram of the Hybrid-INoT introspective workflow

Algorithm 2 Selective multi-candidate rerun

Require: candidate set $\mathcal{Y} = \{y_1, \ldots, y_K\}$, critic scores s_k, rerun budget R
Ensure: filtered candidate $y^\star$

```
Sort 𝒴 in descending order of s_k
Keep the top M ≤ K candidates
for each y_m in the retained set do
    q_m ← ExternalVerify(y_m)
    if q_m = pass then
        return y_m
    end if
end for
if R > 0 then
    Construct a new prompt only from verified failure signals
    return SelectiveRerun(R − 1)
end if
return the candidate with the highest score
```

Algorithm 3 Context compression for long tasks

Require: context $\mathcal{C}_0$, length threshold τ_{len}
Ensure: compressed context $\hat{\mathcal{C}}$

```
if |𝒞_0| ≤ τ_len then
    return 𝒞_0
end if
Split 𝒞_0 into semantic blocks
Score blocks by task relevance and density of verifiable signals
Remove duplicates, low-information logs, and secondary documentation
Retain API signatures, failing traces, editable files, and task constraints
return Ĉ
```

Algorithm 4 Code self-check and maintainability scoring

Require: patch or code answer y
Ensure: verification label v, maintainability score M

```
t ← RunTests(y)
s ← RunStaticAnalysis(y)
cc ← CyclomaticComplexity(y)
dup ← DuplicationRate(y)
warn ← LintWarnings(y)
doc ← DocumentationCoverage(y)
M ← w_cc(1 − c̃c) + w_dup(1 − d̃up) + w_warn(1 − w̃arn) + w_doc d̃oc
if t = pass and s = pass then
    v ← pass
else
    v ← fail
end if
return (v, M)
```

4.5 Metric system

For a heterogeneous benchmark suite, aggregate quality is defined as

$$Q = \sum_{b=1}^{B} w_b Q_b, \qquad \sum_{b=1}^{B} w_b = 1, \tag{7}$$

where $Q_b \in [0, 1]$ is normalized quality on benchmark or task class b and w_b is a predeclared weight.

The main efficiency metrics are then

$$U_{\text{tok}} = \frac{Q}{T}, \tag{8}$$

$$Q_{\$} = \frac{Q}{C_{\text{inf}}}, \tag{9}$$

$$VCR = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}[\texttt{tests}_i = 1 \wedge \texttt{static}_i = 1 \wedge \texttt{security}_i = 1], \tag{10}$$

$$MAS = \frac{1}{N} \sum_{i=1}^{N} \mathit{Success}_i \cdot \mathit{Verified}_i \cdot M_i. \tag{11}$$

Maintainability is estimated with normalized proxy features:

$$M_i = w_{cc}(1 - \widetilde{CC}_i) + w_{dup}(1 - \widetilde{Dup}_i) + w_{warn}(1 - \widetilde{Warn}_i) + w_{doc}\widetilde{Doc}_i, \tag{12}$$

where $\widetilde{CC}_i$ is normalized cyclomatic complexity [34], $\widetilde{Dup}_i$ is duplication, $\widetilde{Warn}_i$ is the normalized number of static-analysis warnings, and $\widetilde{Doc}_i$ is normalized documentation coverage.

To quantify workflow reliability, the paper also uses

$$SCR = \frac{\#\{\text{incorrect candidates caught by self-check}\}}{\#\{\text{all incorrect candidates}\}}, \tag{13}$$

$$RFR = \frac{\#\{\text{patches without post-hoc rollback}\}}{\#\{\text{applied patches}\}}, \tag{14}$$

where SCR is self-check recall and RFR is rollback-free rate.

5 Reproducible Experimental Protocol

5.1 Prompt-length scale and task classes

To avoid evaluation at a single arbitrary context size, we use the following scale:

$$\textit{short}: \quad |\mathcal{C}_0| \leq 512,$$

$$
\begin{aligned}
\textit{medium}: &\quad 512 < |\mathcal{C}_0| \leq 2048, \\
\textit{long}: &\quad 2048 < |\mathcal{C}_0| \leq 8192, \\
\textit{very long}: &\quad |\mathcal{C}_0| > 8192.
\end{aligned}
$$

Tasks are divided into six classes: *single-step*, *multi-step*, *tool-using*, *coding*, *debugging*, and *document reasoning*.

Table 2 Task classes and expected behavior of the proposed architecture

Task class	Main mode	Expected behavior
single-step	internal	One candidate, lightweight critique, no external tools; introspection is minimized to avoid losing to the single-agent baseline on token cost.
multi-step	internal	The planner structures the task, the worker proposes multiple hypotheses, and the critic detects omissions.
tool-using	hybrid	The policy controller selects allowed tools; self-check is mandatory before write or execution actions.
coding	hybrid	The worker produces a patch, and self-check runs tests, static analysis, and maintainability scoring.
debugging	hybrid	Failing traces and relevant files are compressed first, followed by multiple repair candidates and a selective rerun.
document reasoning	internal	Memory/compression removes long-context noise; the critic checks completeness and contradictions.

5.2 Datasets and synthetic-but-reproducible extensions

The core protocol includes three public datasets:

1. **HumanEval** for short code-generation tasks [18];
2. **MBPP** for basic natural-language programming tasks [19];
3. **SWE-bench** for defect-resolution tasks in real repositories [16].

These are complemented by two *synthetic but reproducible* families because public benchmarks do not sufficiently cover long document context and controlled tool-use scenarios:

1. **Synthetic Tool-Use Suite**: tasks generated from a finite grammar of file, terminal, and structured-artifact operations with deterministic oracles.
2. **Synthetic Document Reasoning Suite**: tasks created from publicly permitted technical texts for version comparison, contradiction detection, brief generation, and constraint checking, all with fixed generators and seeds.

5.3 Baseline configurations and statistics

We compare four regimes: **B0 Single-Large**, **B1 Single-Large+Self-Refine**, **B2 Classical-MAS**, and **B3 Hybrid-INoT**. Additional ablations are *no-critic*, *no-compression*, *no-self-check*, *large-model-only*, and *always-external*. For stochastic settings, each configuration is run with at least five random seeds on HumanEval, MBPP, and the synthetic suites; issue-resolution settings may use at least three runs provided that task IDs and seeds are fully disclosed.

The report includes mean values with 95% confidence intervals, bootstrap confidence intervals with at least 10 000 resamples over tasks, McNemar tests for paired binary success, Wilcoxon signed-rank tests for paired token, cost, and latency comparisons, and Holm–Bonferroni correction for multiple comparisons. Closed API models without immutable snapshot identifiers are placed in an observational appendix rather than mixed into the reproducible track.

6 Economic Analysis and TCO

6.1 Total-cost-of-ownership formulation

For a monthly deployment horizon, total cost of ownership is defined as

$$TCO_{\text{month}} = C_{\text{inf}} + C_{\text{hw}} + C_{\text{energy}} + C_{\text{storage}} + C_{\text{ops}}, \tag{15}$$

with

$$C_{\text{hw}} = \frac{CapEx_{\text{cpu}} + CapEx_{\text{gpu}} + CapEx_{\text{ram}}}{L_{\text{amort}}}, \tag{16}$$

$$C_{\text{energy}} = P_{\text{avg}} \cdot h_{\text{month}} \cdot c_{\text{kWh}}, \tag{17}$$

$$C_{\text{ops}} = C_{\text{monitoring}} + C_{\text{maintenance}} + C_{\text{human-review}}. \tag{18}$$

6.2 Synthetic-but-reproducible deployment scenario

Table 3 presents a *synthetic* rather than empirically measured scenario. Its purpose is not to claim a realized gain but to show how architectures should be compared under explicitly declared assumptions.

Under these declared assumptions, Hybrid-INoT reduces monthly synthetic TCO by 23.4%. This is not an empirical measurement from a deployed production system; it is a reproducible worked example.

6.3 When the architecture is beneficial and when it is not

The proposed architecture is most likely to be economically advantageous when tasks share a long common context that would otherwise be replayed across multiple external roles, when a substantial fraction of failures can be caught cheaply before a full rerun of the large model, when true role parallelism is not the main speed advantage, and when helper models or external verifiers are reliable enough to reduce ρ in Eq. (6). It may be disadvantageous under the opposite conditions: very short *single-step* tasks,

Table 3 Synthetic but reproducible monthly TCO example for a deployment stand

Cost item / assumption	Classical-MAS	Hybrid-INoT
Tasks per month	10,000	10,000
Expected large-model input tokens / task	22,000	9,000
Expected large-model output tokens / task	3,200	1,800
Expected helper-model tokens / task	0	1,800
Synthetic inference cost, USD / month	664	497
CPU/RAM server amortization, USD / month	220	220
GPU amortization, USD / month	360	180
Storage and retained artifacts, USD / month	20	20
Energy, USD / month	70	35
Operations and human review, USD / month	300	300
Total synthetic TCO, USD / month	1,634	1,252

large gains from genuine parallelism, weak role-following behavior, or strict security settings that require each tool and privilege boundary to be isolated in a separate process.

7 Discussion

7.1 Why introspection of thought matters specifically for AI agents

For a one-shot chat answer, introspection may only improve the final text. In AI agents, its role is broader: intermediate decisions affect tool choice, future context size, the number of later iterations, and therefore the cost of the entire trajectory. Introspection thus acts both as a quality-control mechanism and as a cost-control mechanism.

7.2 Code reliability, self-check, and maintainability

The key engineering principle of this paper is to reject the assumption that a plausible textual answer automatically implies a reliable code change. Verification must remain external and observable: tests, compilers, linters, static analyzers, and security checks. This is consistent with recent work on compiler feedback and static analysis in code generation [23, 24]. Maintainability is treated as a first-class objective as well. A solution that passes tests only by sharply increasing cyclomatic complexity, duplication, or warning counts should not count as a full success.

7.3 Limitations and threats to validity

The approach has several important limitations. Internal role text does not guarantee a faithful explanation of the model's latent computation [9]. For tasks with genuinely independent roles, an external multi-agent design may remain preferable. Context compression can remove rare but critical evidence if relevance selection is poorly designed.

Maintainability metrics depend on the chosen static proxies and do not replace full architectural review. Finally, closed API models drift over time; without immutable snapshot identifiers they are not suitable for the strict reproducible track.

8 Conclusion

The paper presents a scientifically defensible reformulation of evaluating role segregation in language models via introspection of thought. The revised version relies only on real and citable academic or official primary sources, avoids treating unconfirmed architectural claims as established facts, formalizes role introspection as a mechanism for managing quality, tokens, cost, and reliability, and introduces a reproducible evaluation protocol together with a synthetic TCO calculation under explicit assumptions.

References

[1] H. Sun and S. Zeng, "Introspection of Thought Helps AI Agents," arXiv preprint arXiv:2507.08664, 2025.

[2] A. J. Ko, R. DeLine, and G. Venolia, "Information Needs in Collocated Software Development Teams," in *Proceedings of the 29th International Conference on Software Engineering*, 2007, pp. 344–353.

[3] C. Parnin and S. Rugaber, "Resumption Strategies for Interrupted Programming Tasks," *Software Quality Journal*, vol. 19, no. 1, pp. 5–34, 2011.

[4] M. K. Goncalves, C. R. B. de Souza, and V. M. Gonzalez, "Collaboration, Information Seeking and Communication: An Observational Study of Software Developers' Work Practices," *Journal of Universal Computer Science*, vol. 17, no. 11, pp. 1529–1550, 2011.

[5] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24824–24837.

[6] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," arXiv preprint arXiv:2210.03629, 2022.

[7] A. Madaan *et al.*, "Self-Refine: Iterative Refinement with Self-Feedback," arXiv preprint arXiv:2303.17651, 2023.

[8] N. Shinn *et al.*, "Reflexion: Language Agents with Verbal Reinforcement Learning," arXiv preprint arXiv:2303.11366, 2023.

[9] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting," arXiv preprint arXiv:2305.04388, 2023.

[10] G. Weiss, *Multiagent Systems: A Modern Approach to Distributed Artificial Intelligence*. Cambridge, MA, USA: MIT Press, 1999.

[11] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, UK: Wiley, 2009.

[12] S. Hong *et al.*, "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," arXiv preprint arXiv:2308.00352, 2023.

[13] C. Qian *et al.*, "ChatDev: Communicative Agents for Software Development," arXiv preprint arXiv:2307.07924, 2023.

[14] T. Guo *et al.*, "Large Language Model based Multi-Agents: A Survey of Progress and Challenges," arXiv preprint arXiv:2402.01680, 2024.

[15] C. Qian *et al.*, "Scaling Large Language Model-based Multi-Agent Collaboration," arXiv preprint arXiv:2406.07155, 2024.

[16] C. E. Jimenez *et al.*, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," arXiv preprint arXiv:2310.06770, 2023.

[17] J. Yang *et al.*, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," arXiv preprint arXiv:2405.15793, 2024.

[18] M. Chen *et al.*, "Evaluating Large Language Models Trained on Code," arXiv preprint arXiv:2107.03374, 2021.

[19] J. Austin *et al.*, "Program Synthesis with Large Language Models," arXiv preprint arXiv:2108.07732, 2021.

[20] H. Jiang *et al.*, "LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression," arXiv preprint arXiv:2310.06839, 2023.

[21] Z. Pan *et al.*, "LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression," arXiv preprint arXiv:2403.12968, 2024.

[22] H. Jia, E. T. Barr, and S. Mechtaev, "Compressing Code Context for LLM-based Issue Resolution," arXiv preprint arXiv:2603.28119, 2026.

[23] Z. Bi *et al.*, "Iterative Refinement of Project-Level Code Context for Precise Code Generation with Compiler Feedback," arXiv preprint arXiv:2403.16792, 2024.

[24] C.-Y. Su and C. McMillan, "Do Code LLMs Do Static Analysis?," arXiv preprint arXiv:2505.12118, 2025.

[25] M. Sepidband, H. Taherkhani, S. Wang, and H. Hemmati, "Enhancing LLM-Based Code Generation with Complexity Metrics: A Feedback-Driven Approach,"

arXiv preprint arXiv:2505.23953, 2025.

[26] DeepSeek-AI *et al.*, "DeepSeek-V3 Technical Report," arXiv preprint arXiv:2412.19437, 2024.

[27] DeepSeek-AI *et al.*, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," arXiv preprint arXiv:2501.12948, 2025.

[28] Z.ai, "GLM-4.5: Agentic, Reasoning, and Coding Foundation Models," arXiv preprint arXiv:2508.06471, 2025.

[29] OpenAI, "Introducing ChatGPT agent: bridging research and action," Jul. 17, 2025. [Online]. Available: https://openai.com/index/introducing-chatgpt-agent/

[30] OpenAI, "ChatGPT agent System Card," Jul. 17, 2025. [Online]. Available: https://openai.com/index/chatgpt-agent-system-card/

[31] OpenAI, "Agents SDK," 2026. [Online]. Available: https://platform.openai.com/docs/guides/agents-sdk/

[32] OpenAI, "A Practical Guide to Building Agents," 2026. [Online]. Available: https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf

[33] ISO/IEC 25010:2023, "Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model," International Organization for Standardization, 2023.

[34] T. J. McCabe, "A Complexity Measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.